

HW01 : Codex 最初の一步 (Mac)

動画撮影時（9月初め）から変更があったり、動画内で詰まったりすることが多かったため、「動画からの変更点」が多くあります。注意してください。

この宿題では、Codexを使って次の作業を行います。

1. Codexのモデル、Effort、Speed、Previewを試す
2. 模擬教科書PDFの第10章をまとめる
3. レポート作成とデータ処理に使うソフトを準備する
4. 第10章のデータを最新版に更新してグラフを作る
5. MarkdownからWordとHTMLのレポートを作る
6. 第10章のスライドを作る
7. 第10章のクイズゲームを作る（オプション）
8. レポートとスライド（オプションでクイズ）をGoogle Classroomへ提出する

💡 補足：Claudeを使う場合

- 基本的な作業手順はCodexとほぼ一緒です。
- ChatGPT（Codex）アプリの代わりに、Claudeデスクトップアプリをインストールして使用します。
- 後半でVS Codeをインストールした後は、Codex拡張機能の代わりに Claude拡張機能（Claude extension）をインストールします。

締切と提出物

締切：10月13日（火） 23:59

Google Classroom（クラスコード `jinlu455`）の課題「HW01」に、次のファイルをアップロードします。

- レポート（Word、`.docx`）

- スライド（PDF、`.pdf`）
- クイズ（HTML、`.html`）：オプション

ファイル名の付け方は、最後の「10. Google Classroomへ提出する」を参照してください。

目次

HW01：Codex 最初の一步（Mac）

締切と提出物

1. Codexを開く
2. Codexの基礎概念を試す
3. 第10章をまとめる
4. レポート作成に使うソフトをインストールする
5. Visual Studio Codeを準備する
6. 第10章のデータを更新する
7. 第10章のレポートを作る
8. 第10章のスライドを作る
9. 第10章のクイズゲームを作る（オプション）
10. Google Classroomへ提出する

1. Codexを開く

動画での修正点

5.のフォルダ作成を先にやりました。

1. ChatGPTデスクトップアプリをMacにインストールします。
2. アプリを起動し、ChatGPTアカウントでサインインします。
3. アプリの **Codex** を選びます。
4. 新しいチャットを開き、実行環境で **Local** を選びます。
5. 書類（Documents）フォルダの中に **金融論** フォルダを作り、その中に **HW01** フォルダを作ります。
6. Codexで **書類（Documents）/金融論/HW01** フォルダを開きます。
7. フォルダへのアクセスを求められたら許可します。
8. 配布された **模擬教科書/moc_text_claude.pdf** を **HW01** フォルダへ入れます。

第4節までは、ChatGPTデスクトップアプリのCodexを使います。

Enter／Returnで改行する設定

長い指示を複数行で書いている途中で誤送信しないよう、Codex appの入力方法を変更します。

- **Enter**／**Return**：改行
- **Command + Enter**：入力した指示を送信

Codex appの設定で、**Enter**／**Return** だけでは送信せず、**Command + Enter** で送信するように変更します。

2. Codexの基礎概念を試す

Codexは、入力された指示をAIモデルが考え、必要に応じてファイルの読み書きやコマンドの実行を行う仕組みです。まず、画面に表示される設定の意味を確認します。

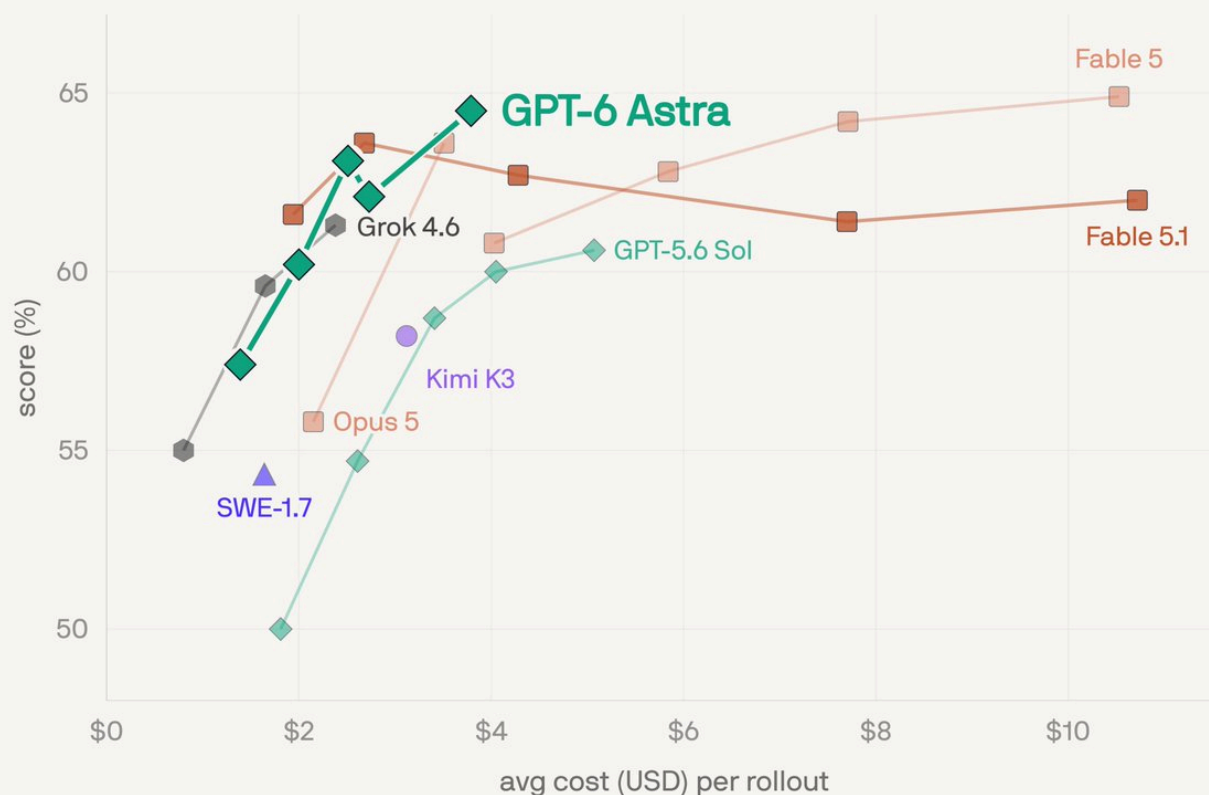
Model、Effort、Speed

- **Model (モデル)**：指示を読んで考え、回答やファイルを作るAIの種類です。
 - **Astra**：複雑な推論、coding、調査、複数toolを使う長い作業に向く、最も高性能なモデルです。
 - **Sol**：複雑で曖昧な課題、深い分析、仕上がりを重視する作業に向きます。
 - **Luna**：手順や完成形が明確な、短く反復的な作業に向きます。
- **Effort (思考量)**：同じモデルに、どの程度深く考えさせるかを指定します。`Light`、`Medium`、`High`、`Extra High`、`Max` などがあり、高くするほど複雑な検討に向きますが、完了までの時間と使用量も増えます。回答文の長さを指定する設定ではありません。通常は `Medium` から始めます。
- **Speed (処理速度)**：`Standard` は通常速度です。`Fast` は対応モデルを速く動かしますが、より多くのクレジットを使います。この宿題では `Standard` を使います。

動画（ビデオ）との違い：Astraの追加

動画収録時に説明したSol、Terra、Lunaに加えて、現在は一番性能の高いAstraが追加されました。また、最新版のGPT 6では、Terraが廃止になっています。実際の作業では、画面に表示されているモデルから選びます。

FrontierCode 1.1 Extended Score vs Cost (\$)



FrontierCode 1.1 ExtendedにおけるModel・Effort別のscoreと1 rollout当たり平均cost

出典：Cognition、[FrontierCode 1.1 Extendedのbenchmark結果](#)

Previewとは

Preview (プレビュー) は、作成したファイルを完成時に近い見た目で表示して確認する機能です。プレビューそのものが別の提出ファイルになるわけではありません。

- Markdownでは、`#`や`-`を含む元のテキストが、見出しや箇条書きとしてどのように表示されるかを確認できます。
- PDFでは、ページの配置、文字、表、図などを確認できます。
- HTMLでは、ブラウザで表示されるページや、ボタンなどの動作を確認できます。

Permissions（権限）

Permissionsは、Codexがファイルの編集、commandの実行、internetの利用などを、どこまで自動で行えるかを定める設定です。入力欄の下にある権限のメニューから選びます。

- **Ask for approval**：開いているworkspace内では作業を進め、internetの利用やworkspace外の操作が必要なときは確認を求めます。この宿題ではこれを使います。
- **Approve for me**：追加の権限が必要な操作を別のAIが確認します。自分で確認する回数は減りますが、判断を誤る可能性もあります。
- **Full access**：確認なしで広い範囲のファイルやinternetへアクセスできます。この宿題では使いません。

承認を求める画面が出たら、何を行うのかを読んでから許可します。内容が分からない場合は許可せず、Codexに説明を求めてください。詳しくは[OpenAI DocsのPermissions](#)を参照してください。

Codexの利用制限（5時間制限と1週間制限）

Codex（ChatGPT Plus等）には、計算資源の公平な利用のために2段階の利用制限が設けられています。

- 5時間ローリング制限：
 - 過去5時間以内の推論・トークン消費量を対象とする短期の制限です。
 - スライディング形式のため、5時間前に使った分から順次回復します。短時間に重い作業を集中させると一時的に利用が制限されます。
- 1週間制限：
 - 1週間あたりの全体的な利用可能枠です。週ごとのリセット日に更新されます。
- 課題を進める上での注意点：
 - 締め切り直前に一気に進めようとする、5時間制限にひっかるかもです。
 - Effort は、通常は小さめに。

エクササイズ：MarkdownとPDFをPreviewする

Codexに次のように指示します。

```
preview_test.mdを作って。タイトルを「Codex設定テスト」とし、今日確認したModel、Effort、Speedを箇条書き
```

2.のファイル名が間違っていました。

1. 作成された `preview_test.md` を開き、タイトルと箇条書きが整形されて表示されることを確認します。
2. `moc_text_claude.pdf` を開き、PDFのページが表示されることを確認します。
3. Previewは見た目の確認、元の `.md` や `.pdf` は実際に保存されているファイルだと区別します。

3. 第10章をまとめる

Codexに次のように指示します。

`moc_text_claude.pdf`の第10章「貨幣・信用創造とマネースtock統計」を読み、内容を簡単にまとめた`chapter10_`

最後に、ChatGPTデスクトップアプリで `chapter10_summary.md` を開き、Markdownの内容を確認します。

4. レポート作成に使うソフトをインストールする

最初に、文書形式について二つの言葉を確認します。

- **Markdown** : 文章の見出しを `#`、箇条書きを `-` のような簡単な記号で表す書き方です。Wordのように画面上で文字を飾るのではなく、内容と構造をテキストとして記録します。ファイル名の最後は `.md` になります。この宿題で作る `chapter10_summary.md` や `chapter10_report.md` がMarkdownファイルです。
- **Marp** : Markdownを使ってスライドを作る仕組みです。Markdownの中でスライドの区切りや見出しを書き、同じファイルからスライドのプレビューやPDFを作れます。この宿題では `chapter10_slides.md` から `chapter10_slides.pdf` を作ります。

インストールするソフトには、次の役割があります。

- **Homebrew** : Macにソフトをインストールし、更新や管理を行うための仕組みです。この宿題では、Python、Node.js、Pandocなどの導入に使います。
- **Python** : データを処理するためのプログラミング言語です。Web上の公開データを取得し、必要な期間や項目を選び、合計や増加率を計算して、グラフを作るときに使います。この宿題では、第10章のデータとグラフを最新のものへ更新するために使います。
- **Node.js** : JavaScriptというプログラミング言語で作られた道具を、Mac上で動かすための実行環境です。この宿題では、Marpの変換機能を動かし、Markdownで書いたスライドをPDFへ変換するために使います。自分でJavaScriptを書く必要はありません。

- **Marp CLI** : Markdownで作成したスライドを、HTMLやPDFへ変換するための道具です。Node.js上で動きます。
- **Pandoc** : ある文書形式を別の文書形式へ変換するソフトです。この宿題では、Markdownで書いた一つのレポート原稿から、提出や共有に使いやすいWordファイル（`.docx`）と、ブラウザで読めるHTMLファイル（`.html`）を作ります。
- **Visual Studio Code** : Markdown、プログラム、データなどのファイルを開いて編集するアプリです。左側でフォルダ内のファイルを確認し、本文を自分で直し、Markdownの見た目をプレビューできます。Codexの拡張機能もここで使います。
- **Google Chrome** : Webページを見るためのブラウザです。この宿題では、作成したHTMLレポートとクイズゲームを開いて確認します。また、MarpがスライドをPDFへ変換するときにも、Chromeの機能を使います。

Homebrewとは

Homebrewは、Macへソフトをインストールし、管理するための仕組みです。このような仕組みをパッケージマネージャーと呼びます。通常はWebサイトからインストーラーを探して一つずつ操作しますが、Homebrewでは、ターミナルに `brew install ソフト名` という命令を入力してインストールできます。

この宿題では、CodexがHomebrewの命令を実行し、Python、Node.js、Pandoc、Visual Studio Codeなどを準備します。Homebrewを一度入れておけば、後からソフトの追加や更新を行うときにも利用できます。この段階で命令を暗記する必要はありません。

まず、Codexに次のように指示します。

このMacにHomebrewを入れて。

動画での修正点

Codexに指示しても上手くいかず、手動でインストールしました。

1. Applicationsフォルダ内の、Utilitiesフォルダにあるターミナルを開きます。
2. 以下のHomebrew公式サイトにあるコマンドを、ターミナルにコピーします。動画では、URLをCodexに聞いて教えてもらいました。
`https://brew.sh/`
3. ターミナルがコマンドの実行許可を求めたら、許可します。
4. ターミナルに `Password:` と表示されたら、Macへログインするときのパスワードを入力し、ReturnもしくはEnterキーを押します。入力中は画面に文字が表示されません。

5. `Press RETURN to continue` と表示されたら、ReturnかEnterキーを押します。
6. Command Line Toolsのインストール画面が表示されたら、インストールします。これは時間がかかります。
7. Homebrewのインストールが終わったら、Codexに次のように指示します。

Homebrewがインストールされたことを確認して

次に、次のように指示します。

Homebrewを使ってPython、Node.js、Pandocをインストールして。それぞれのバージョンを表示して、使えることも確認して。

続けて、Marp CLIをインストールします。

npmを使ってMarp CLIをグローバルインストールして。marp --versionを実行し、使えることも確認して。

次に、データ処理に使うPython環境を準備します。HW01 フォルダが大きくならないように、仮想環境はHW01の外に作ります。

書類 (Documents) /金融論/finance-envにPythonの仮想環境を作って。
その仮想環境にpandas、matplotlib、requestsをインストールし、すべて読み込めることを確認して。
以後、この宿題のPython作業ではfinance-envを使って。

finance-env は、Python本体と、pipで入れたpandasなどのライブラリをまとめたフォルダです。

続けて、次のように指示します。

Homebrewを使ってVisual Studio Codeをインストールして。

動画での修正点

Visual Studio Codeのログインに手こずりました。Githubのアカウントを皆さんは持っていないと思うので、Googleなどを使ってログインしてください。なんかガチャガチャやってたらログインできました。VS code上でgithubでログインと出たら、無視してください。

5. Visual Studio Codeを準備する

Visual Studio Codeを開き、書類 (Documents) / 金融論/HW01 フォルダを開きます。

拡張機能画面 (`Cmd + Shift + X`) から、次をインストールします。

- `openai.chatgpt` (Codex)
- `marp-team.marp-vscode` (Marp)

Codexアイコンをクリックし、画面右側にCodexのchat windowを開きます。

VS Codeには標準AI機能としてGitHub Copilotもありますが、この宿題では使いません。

Visual Studio Codeで `chapter10_summary.md` を開き、Markdownの中身とプレビューを確認します。

動画で触れなかった点：保存の印

ファイルを編集すると、タブの名前の右側の × が ● に変わります。変更がまだ保存されていない印です。

`Cmd + S` で保存すると ● が消えます。Codexやターミナルが読むのは保存済みのファイルなので、● が付いたまま作業を頼むと、保存前の古い内容が使われます。Codexに頼む前と、提出するファイルを作る前に、● が残っていないか確かめます。

ここからは、Visual Studio CodeのCodexを使います。

ファイルの行をCodexに見せる

直してほしい箇所をCodexへ正確に伝えるには、次のようにします。

1. ファイルを開き、見てほしい行をマウスで選びます。
2. 選んだ部分を右クリックし、`Add to Codex Thread` を選びます。
3. 続けて「ここを短くして」「この計算を説明して」などを入力します。

ファイル全体を見せる場合は、ファイルのタブを右クリックして `Add File to Codex Thread` を選びます。

よく使う場合は、`Add to Codex Thread` のkeyboard shortcutを作ると便利です。作成方法はCodexに聞いてください。

6. 第10章のデータを更新する

第10章の図10-3「マネタリーベースとマネーストックの伸び」を、日本銀行の最新の公式データで作り直します。

Codexに次のように指示します。

moc_text_claude.pdfの図10-3「マネタリーベースとマネーストックの伸び」を、日本銀行の最新の公式データで更新
Pythonは書類（Documents）/金融論/finance-envの仮想環境を使って。
データ、取得・作図コード、更新したグラフ、データの出所と取得日を記録したMarkdownをdata_updateフォルダにま

`data_update` フォルダの中のグラフと出所を確認します。

動画での修正点

vscode-pdfという機能拡張も入れました。

7. 第10章のレポートを作る

第10章の要約と更新したグラフを使い、Markdownの原稿からWordとHTMLを作ります。

Codexに次のように指示します。

moc_text_claude.pdf、chapter10_summary.md、data_updateフォルダの更新した図表を使って、第10章の短い
原稿はchapter10_report.md、AIに依頼した作業の記録はAI_LOG.mdにまとめて。
Pandocを使ってoutputフォルダにchapter10_report.docxとchapter10_report.htmlも作り、表、図、出所、リ

次のファイルを開いて確認します。

- `chapter10_report.md`
- `AI_LOG.md`
- `output/chapter10_report.docx`
- `output/chapter10_report.html`

8. 第10章のスライドを作る

Codexに次のように指示します。

moc_text_claude.pdf、chapter10_summary.md、data_updateフォルダの更新した図表を使って、第10章のスライド形式のchapter10_slides.mdとchapter10_slides.pdfを作り、表示も確認して。
 marpコマンドは--no-stdinと--allow-local-filesを付けて実行して。

Visual Studio Codeで `chapter10_slides.md` を開き、Markdownの中身とスライドのプレビューを確認します。

9. 第10章のクイズゲームを作る（オプション）

📺 動画での修正点

動画内で逸見さんのCodexの利用量制限のため、クイズゲームは中止しました。皆さんで試してみてください。宿題提出では必須ではないことにします。

Codexに次のように指示します。

moc_text_claude.pdfとchapter10_summary.mdを使って、第10章を復習する四択クイズゲームを作って。
 10問をquiz.htmlにまとめて。CSS、JavaScript、画像もすべてquiz.htmlの中に入れ、ほかのファイルやインターネット
 quiz.htmlだけを別のフォルダにコピーして開き、動作も確認して。

10. Google Classroomへ提出する

締切：10月13日（火） 23:59

提出するのは次のファイルです。クイズはオプションです。作成したファイルをコピーして、以下のような名前に変更したものを作ってください。

提出物	元のファイル	提出するファイル名
レポート	<code>output/chapter10_report.docx</code>	<code>学籍番号_好きな曲.docx</code>
スライド	<code>chapter10_slides.pdf</code>	<code>学籍番号_好きな漫画.pdf</code>
クイズ（オプション）	<code>quiz.html</code>	<code>学籍番号_好きな映画.html</code>

ファイル名の後半には、自分の好きな曲、漫画、映画の名前を入れます。例えば、次のようになります。

12345678_レミオロメン3月9日.docx

12345678_僕らはみんな河合荘.pdf

クイズ（オプション） 12345678_ヴァイオレットエヴァーガーデン.html

ファイル名には空白と記号（`/`、`:`、`?` など）を入れません。

ファイルを提出する

1. Google Classroomにクラスコード `jinlu455` で参加します。
2. 課題「HW01」を開きます。
3. 「+ 追加または作成」から「ファイル」を選び、名前を変えたファイルを選びます。
4. レポートとスライド（クイズを作った場合はクイズも）が添付されていることを確認し、「提出」を押します。